
```

% =====
% Waveguide Propagation Loss Extraction using Cut-Back Method
% Clean Corrected Version
% =====

clear;
clc;
close all;

% Stop exactly at error line if any error occurs
dbstop if error

% =====
% CHECK IF REGRESSION TOOLBOX EXISTS
% =====
Error_Intervals = exist('regress','file');

FONTSIZE = 13;

% =====
% DEVICE INFORMATION
% =====
deviceName = 'Spiral waveguide (TM)';

% Wavelength of interest
lambda0 = 1.55e-6;

% =====
% FILE NAMES
% =====
files = { ...
    'LukasC_SpiralWG0kTM_1293.mat', ...
    'LukasC_SpiralWG5kTM_1296.mat', ...
    'LukasC_SpiralWG10kTM_1294.mat', ...
    'LukasC_SpiralWG30kTM_1295.mat' ...
};

% Number of DUTs / lengths
Num = [0, 0.5, 1.0, 3.0];

% Detector port number
PORT = 2;

% =====
% DROPBOX URLS
% =====
url = { ...
    'https://www.dropbox.com/s/anlo8zmydrji1f8/LukasC_SpiralWG0kTM_1293.mat?dl=1', ...
    'https://www.dropbox.com/s/yvdfgl8qoq3d0fz/LukasC_SpiralWG5kTM_1296.mat?dl=1', ...
    'https://www.dropbox.com/s/rossgslht5r5cq7/LukasC_SpiralWG10kTM_1294.mat?

```

```

dl=1', ...
    'https://www.dropbox.com/s/vj4uf6u59h9vt60/LukasC_SpiralWG30kTM_1295.mat?
dl=1' ...
    };

% =====
% DOWNLOAD FILES IF NOT PRESENT
% =====
if ~exist(files{1}, 'file')

    disp('Loading files from Dropbox')

    for i = 1:length(files)

        fprintf('Downloading file %d of %d...\n', i, length(files));

        websave(files{i}, url{i});

    end

else

    disp('Loading files from local disk')

end

% =====
% INITIALIZE ARRAYS
% =====
amplitude = [];
amplitude_poly = [];
LegendText = {};

% =====
% PLOT RAW DATA + POLYFIT
% =====
figure;

for i = 1:length(files)

    % Load MAT file
    load(files{i});

    % Force wavelength into column vector
    lambda = scandata.wavelength(:);

    % Extract detector data
    temp_power = scandata.power(:, PORT);

    % Ensure column vector
    temp_power = temp_power(:);

    % Store
    amplitude(:, i) = temp_power;

```

```

% Plot raw data
plot(lambda*1e6, amplitude(:,i));
hold on;

% Polynomial fitting
xfit = (lambda - mean(lambda))*1e6;

p = polyfit(xfit, amplitude(:,i), 4);

amplitude_poly(:,i) = polyval(p, xfit);

% Plot fitted data
plot(lambda*1e6, amplitude_poly(:,i), 'LineWidth',2);

% Legends
LegendText{2*i-1} = ...
    ['raw data: ' strcat(files{i}, '_', '\_')];

LegendText{2*i} = ...
    ['fit data: ' strcat(files{i}, '_', '\_')];

end

title(['Optical spectra for the ' deviceName ' test structures']);

xlabel('Wavelength (\nmum)');

ylabel('Optical Power (dBm)');

legend(LegendText, 'Location', 'South');

axis tight;

set(gca, 'FontSize', FONTSIZE)

% =====
% LINEAR REGRESSION AT lambda0
% =====

% Find nearest wavelength index
[~, index] = min(abs(lambda - lambda0));

% Linear regression
A = [ones(length(Num),1) Num] \ amplitude_poly(index,:);

% =====
% PLOT CUT-BACK FIT
% =====

figure;

plot(Num, amplitude(index,:), 'x');
hold on;

```

```

plot(Num, amplitude_poly(index,:), 'o', 'MarkerSize', 7);

plot(Num, A(1) + Num*A(2), 'LineWidth', 3);

legend( ...
    'raw data at lambda0', ...
    'polyfit of raw data', ...
    'linear regression of polyfit');

xlabel('Waveguide length (cm)');

ylabel('Loss (dB/cm)');

title(['Cut-back method, ' deviceName ...
    ' Loss, at ' num2str(lambda0*1e9) ' nm']);

set(gca, 'FontSize', FONTSIZE)

% =====
% CONFIDENCE INTERVALS
% =====
if Error_Intervals

    [b, bint] = regress( ...
        amplitude_poly(index,:), ...
        [Num' ones(numel(Num), 1)]);

    SlopeError95CI = diff(bint(1, :))/2;

    InterceptError95CI = diff(bint(2, :))/2;

    a = annotation('textbox', [.2 .2 .1 .1], ...
        'String', ...
        { ...
        ['Fit results, with 95% confidence intervals:'], ...
        ['slope = ' num2str(A(2), '%0.2g') ...
        ' +/- ' num2str(SlopeError95CI, '%.01g') ' dB/cm'], ...
        ['intercept = ' num2str(A(1), '%0.3g') ...
        ' +/- ' num2str(InterceptError95CI, '%.02g') ' dB'] ...
        });

    set(a, 'FontSize', FONTSIZE);

    disp(['Cut-back method, ' deviceName ...
        ' Loss at ' num2str(lambda0*1e9) ...
        ' nm = ' num2str(-A(2), '%0.2g') ...
        ' +/- ' num2str(SlopeError95CI, '%.01g') ...
        ' dB/cm'])

else

    disp('Skipping fitting error estimations')

    annotation('textbox', [.2 .2 .1 .1], ...

```

```

        'String', ...
        { ...
        ['Fit results:'], ...
        ['slope = ' num2str(A(2), '%0.2g') ' dB/cm'], ...
        ['intercept = ' num2str(A(1), '%0.3g') ' dB'] ...
        });

disp(['Cut-back method, ' deviceName ...
     ' Loss at ' num2str(lambda0*1e9) ...
     ' nm = ' num2str(-A(2), '%0.2g') ...
     ' dB/cm'])

end

% =====
% WAVELENGTH DEPENDENCE OF LOSS
% =====

% Linear regression using raw data
C = [ones(length(Num),1) Num] \ amplitude';

figure

if Error_Intervals

    slope = [];
    slope_int = [];

    lambda_downsampled = lambda(1:100:end);

    amplitude_poly_downsampled = ...
        amplitude_poly(1:100:end,:);

    for i = 1:length(lambda_downsampled)

        [b, bint] = regress( ...
            amplitude_poly_downsampled(i,:)', ...
            [Num' ones(numel(Num),1)]);

        slope(i) = b(1);

        slope_int(i,:) = bint(1,:);

    end

    % Confidence interval region
    X = [lambda_downsampled; flipud(lambda_downsampled)]*1e6;

    Y = [slope_int(:,1); flipud(slope_int(:,2))];

    fill(X, -Y, [0.7 1 1], ...
         'LineStyle','none');

    hold on;

```

```

% Polyfit-based result
plot(lambda_downsampled*1e6, ...
    -slope', ...
    'b', ...
    'LineWidth',3);

% Raw-data result
plot(lambda*1e6, ...
    -C(2,:) ', ...
    'LineWidth',1.5);

legend( ...
    'Loss, 95% Confidence Interval', ...
    'Loss from polyfit', ...
    'Loss from raw data', ...
    'Location','Best');

else

    D = [ones(length(Num),1) Num'] \ amplitude_poly';

    plot(lambda*1e6, ...
        -[D(2,:) ' C(2,:) ']);

    legend( ...
        'Loss from polyfit', ...
        'Loss from raw data', ...
        'Location','Best');

end

axis tight;

yl = ylim;

ylim([0 yl(2)])

title(['Cut-back method, ' deviceName ...
    ' Loss wavelength dependence'])

ylabel('Loss (dB/cm)');

xlabel('Wavelength (\mum)');

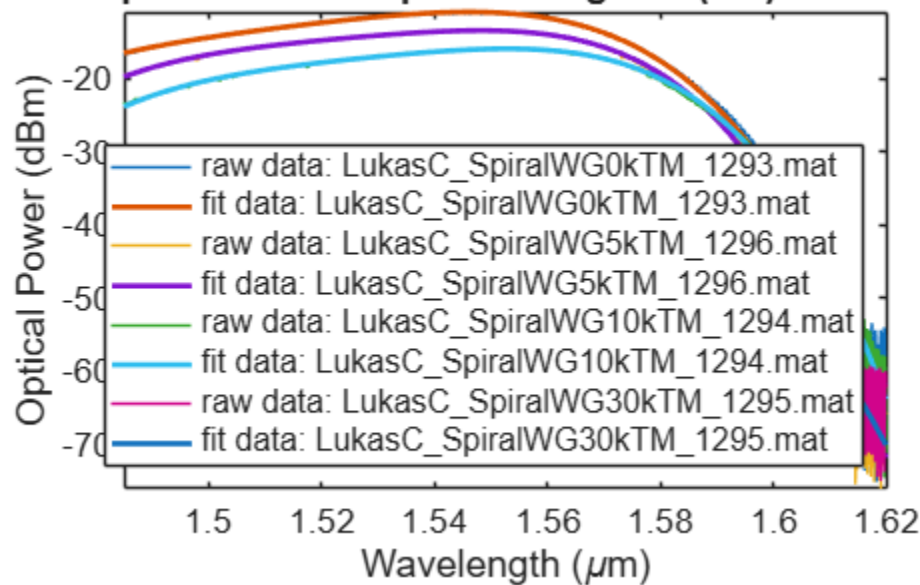
set(gca, 'FontSize', FONTSIZE)

disp('Program completed successfully.')

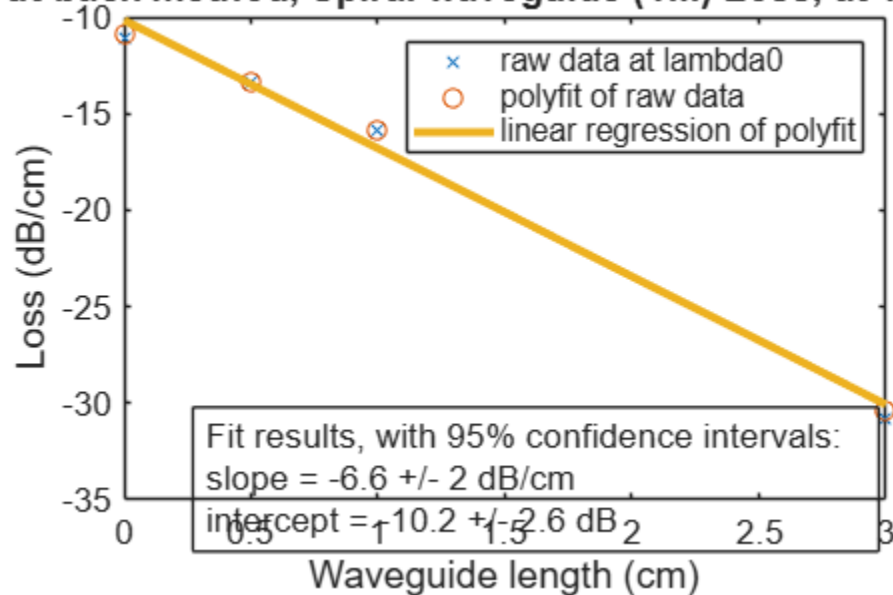
Loading files from local disk
Cut-back method, Spiral waveguide (TM) Loss at 1550 nm = 6.6 +/- 2 dB/cm
Program completed successfully.

```

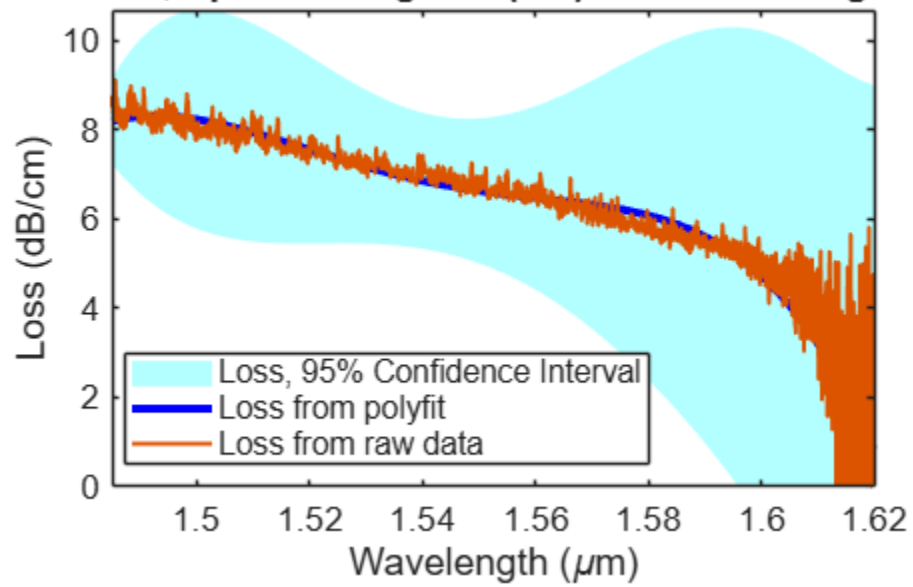
Optical spectra for the Spiral waveguide (TM) test structure



Cut-back method, Spiral waveguide (TM) Loss, at 1550 nm



:k method, Spiral waveguide (TM) Loss wavelength de



Published with MATLAB® R2026a